LAB: EF Core

To transition your application from using a temporary in-memory list (_books) to a real, persistent database, we need to integrate **Entity Framework Core (EF Core)**.

For this guide, we will use **SQLite**. It is lightweight, cross-platform, requires no separate server installation, and is the standard database used in .NET tutorials and labs.

Step 1: Install Required NuGet Packages

Open your terminal inside your project folder and run the following commands to install the EF Core SQLite provider and the Design tools (needed for creating the database):

Bash
```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Design
```

Step 2: Configure the Connection String

Open **appsettings.json** at the root of your project and add a ConnectionStrings block. This tells EF Core where to create the SQLite database file (app.db).

JSON
```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=app.db"
  }
}
```

Step 3: Create the Database Context

The DbContext is the main class that coordinates Entity Framework functionality for your data models.

Create a new folder named **Data** in your project root, and add a file named **AppDbContext.cs**.

C#
```
using Microsoft.EntityFrameworkCore;
using MyMvcApp.Models;

namespace MyMvcApp.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        {
        }

        // This property represents the table in your database
        public DbSet<Book> Books { get; set; }
    }
}
```

Step 4: Register the DbContext in Program.cs

You must tell ASP.NET Core about your AppDbContext so it can be injected into your controllers.
Open **Program.cs** and add the database configuration right before builder.Build();:

C#
```
using Microsoft.EntityFrameworkCore;
using MyMvcApp.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

// Register the AppDbContext with SQLite
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();
```

// ... (Rest of your Program.cs remains the same)

Step 5: The Model (Remains Unchanged)

Your Book.cs model already contains the Id property (which EF Core automatically recognizes as the Primary Key) and the Data Annotations for validation. **Keep it exactly as you have it:**

C#
```
using System.ComponentModel.DataAnnotations;

namespace MyMvcApp.Models
{
    public class Book
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "The Title field is required.")]
        [StringLength(100, MinimumLength = 2, ErrorMessage = "Title must be between 2 and 100 characters.")]
        public string Title { get; set; } = string.Empty;

        [Required(ErrorMessage = "Please provide the author's name.")]
        [StringLength(50, ErrorMessage = "Author name cannot exceed 50 characters.")]
        public string Author { get; set; } = string.Empty;
    }
}
```

Step 6: Update the BooksController

We need to remove the static _books list and replace it with our AppDbContext. Database operations should also be **asynchronous** (async / await) so they don't block the web server while waiting for the database to respond.

Replace the entire contents of **Controllers/BooksController.cs** with this updated code:

C#
```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MyMvcApp.Data;
using MyMvcApp.Models;
```

```
namespace MyMvcApp.Controllers
{
    public class BooksController : Controller
    {
        private readonly AppDbContext _context;

        // The database context is injected here by ASP.NET Core
        public BooksController(AppDbContext context)
        {
            _context = context;
        }

        // GET: Books
        public async Task<IActionResult> Index()
        {
            // Fetch all books from the database asynchronously
            var books = await _context.Books.ToListAsync();
            return View(books);
        }

        // GET: Books/New
        [HttpGet("New")]
        public IActionResult Create()
        {
            return View();
        }

        // POST: Books/New
        [HttpPost("New")]
        [ValidateAntiForgeryToken] // Protects against Cross-Site Request Forgery
        public async Task<IActionResult> Create(Book book)
        {
            // 1. Check Form Validation
            if (!ModelState.IsValid)
            {
                return View(book); // Validation failed, return to form
            }

            // 2. Add to EF Core and Save to Database
            _context.Books.Add(book);
```

```
    await _context.SaveChangesAsync();

    return RedirectToAction(nameof(Index));
}

// GET: Books/{id}/Edit
[HttpGet("{id:int}/Edit")]
public async Task<IActionResult> Edit(int id)
{
    // Find the book in the database
    var book = await _context.Books.FindAsync(id);
    if (book == null)
    {
        return NotFound();
    }
    return View(book);
}

// POST: Books/{id}/Edit
[HttpPost("{id:int}/Edit")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id, Book updatedBook)
{
    if (id != updatedBook.Id)
    {
        return BadRequest();
    }

    // 1. Check Form Validation
    if (!ModelState.IsValid)
    {
        return View(updatedBook); // Validation failed, return to form
    }

    try
    {
        // 2. Update in EF Core and Save to Database
        _context.Update(updatedBook);
        await _context.SaveChangesAsync();
    }
```

```
            catch (DbUpdateConcurrencyException)
            {
                if (!BookExists(updatedBook.Id))
                {
                    return NotFound();
                }
                else
                {
                    throw;
                }
            }

            return RedirectToAction(nameof(Index));
        }

        // Helper method to check if a book exists
        private bool BookExists(int id)
        {
            return _context.Books.Any(e => e.Id == id);
        }
    }
}
```

(Note: Keep your Create.cshtml and Edit.cshtml exactly as you provided them in the prompt. They are perfectly configured to work with this controller.)

Step 7: Create the Database using Migrations

Now that the code is written, you must tell Entity Framework to physically generate the database file and table structure based on your Book model.

Run these two commands in your terminal:

1. **Create the Migration** (This generates C# instructions on how to build the DB):

   Bash

   ```
   dotnet ef migrations add InitialCreate
   ```

2. **Apply the Migration** (This executes the instructions and creates app.db):
   Bash

   dotnet ef database update

Summary of Changes:

- **Dependency Injection:** The BooksController no longer relies on a static list. It now receives AppDbContext via its constructor.

- **Asynchronous Execution:** Methods now return Task<IActionResult> and use await _context.SaveChangesAsync(). This is a crucial best practice for database I/O in web applications.

- **Validation Preservation:** The !ModelState.IsValid blocks remain exactly where you put them. If the data is invalid, the controller skips the EF Core database operations entirely and returns the View.

Code Explanation:

Section 1: appsettings.json (Database Configuration)

JSON

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
```

```
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DefaultConnection": "Data Source=app.db"
  }
}
```

Explanation:

- "Logging": { ... }: Configures how much system information gets printed to the console.
- "AllowedHosts": "*": Specifies which domains can access this application (the asterisk means any host is allowed).
- "ConnectionStrings": {: Begins the section where database connections are securely stored.
- "DefaultConnection": "Data Source=app.db": Defines a connection named "DefaultConnection". The value "Data Source=app.db" tells Entity Framework to create and use a local SQLite database file named app.db in the root of your project.

Section 2: Data/AppDbContext.cs (Entity Framework Context)

C#

```
using Microsoft.EntityFrameworkCore;
using MyMvcApp.Models;

namespace MyMvcApp.Data
{
```

```
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
        {
        }

        public DbSet<Book> Books { get; set; }
    }
}
```

Explanation:

- using Microsoft.EntityFrameworkCore;: Imports the Entity Framework Core library.
- using MyMvcApp.Models;: Imports the folder containing your Book class so it can be used here.
- namespace MyMvcApp.Data: Organizes this file into the Data namespace.
- public class AppDbContext : DbContext: Creates your custom database context class by inheriting from EF Core's base DbContext class.
- public AppDbContext(DbContextOptions<AppDbContext> options) : base(options): The constructor. It accepts configuration options (like the connection string from Program.cs) and passes them (base(options)) to the underlying EF Core DbContext.
- { }: Empty constructor body, as the base class handles the setup.

- public DbSet<Book> Books { get; set; }: Represents the actual database table. EF Core will look at this line and create a table named "Books" where each row represents a Book object.

Section 3: Program.cs (Registering the Database)

C#

```
// (Other using statements)

using Microsoft.EntityFrameworkCore;

using MyMvcApp.Data;


var builder = WebApplication.CreateBuilder(args);


builder.Services.AddControllersWithViews();


builder.Services.AddDbContext<AppDbContext>(options =>

    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));


var app = builder.Build();

// (Rest of the file)
```

Explanation:

- using Microsoft.EntityFrameworkCore; / using MyMvcApp.Data;: Imports necessary libraries and your new Data folder.

- var builder = WebApplication.CreateBuilder(args);: Initializes the application builder.
- builder.Services.AddControllersWithViews();: Adds support for MVC (Model-View-Controller).
- builder.Services.AddDbContext<AppDbContext>(options =>: Registers your AppDbContext with ASP.NET Core's Dependency Injection system so it can be automatically provided to your controllers.
- options.UseSqlite(...): Tells Entity Framework Core to use the SQLite database provider.
- builder.Configuration.GetConnectionString("DefaultConnection"): Reaches into appsettings.json, grabs the string "Data Source=app.db", and passes it to SQLite.
- var app = builder.Build();: Compiles the application configuration.

Section 4: Models/Book.cs (Data Model)

C#

```
using System.ComponentModel.DataAnnotations;


namespace MyMvcApp.Models

{

    public class Book

    {

        public int Id { get; set; }



        [Required(ErrorMessage = "The Title field is required.")]

        [StringLength(100, MinimumLength = 2, ErrorMessage = "Title must be between 2 and 100 characters.")]
```

```
        public string Title { get; set; } = string.Empty;


        [Required(ErrorMessage = "Please provide the author's name.")]
        [StringLength(50, ErrorMessage = "Author name cannot exceed 50 characters.")]
        public string Author { get; set; } = string.Empty;
    }
}
```

Explanation:

- using System.ComponentModel.DataAnnotations;: Imports the library required for validation attributes.
- public class Book: Defines the Book object.
- public int Id { get; set; }: The unique identifier. EF Core automatically makes properties named Id the Primary Key in the database table.
- [Required(...)]: A validation rule dictating that the property directly below it cannot be null or empty. Includes a custom error message.
- [StringLength(...)]: A validation rule dictating the maximum (and optionally minimum) characters allowed. EF Core also uses this to restrict column sizes in the database.
- public string Title { get; set; } = string.Empty;: The property holding the book's title. Initialized to string.Empty to prevent null reference warnings.
- public string Author { get; set; } = string.Empty;: The property holding the author's name.

Section 5: Controllers/BooksController.cs (The Logic)

C#

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MyMvcApp.Data;
using MyMvcApp.Models;

namespace MyMvcApp.Controllers
{
    public class BooksController : Controller
    {
        private readonly AppDbContext _context;

        public BooksController(AppDbContext context)
        {
            _context = context;
        }

        // --- INDEX ---
        public async Task<IActionResult> Index()
        {
            var books = await _context.Books.ToListAsync();
            return View(books);
        }
```

```
// --- CREATE (GET) ---
[HttpGet("New")]
public IActionResult Create()
{
    return View();
}

// --- CREATE (POST) ---
[HttpPost("New")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(Book book)
{
    if (!ModelState.IsValid)
    {
        return View(book);
    }

    _context.Books.Add(book);
    await _context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}
```

```
// --- EDIT (GET) ---
[HttpGet("{id:int}/Edit")]
public async Task<IActionResult> Edit(int id)
{
    var book = await _context.Books.FindAsync(id);
    if (book == null) return NotFound();

    return View(book);
}

// --- EDIT (POST) ---
[HttpPost("{id:int}/Edit")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id, Book updatedBook)
{
    if (id != updatedBook.Id) return BadRequest();

    if (!ModelState.IsValid)
    {
        return View(updatedBook);
    }

    _context.Update(updatedBook);
```

```
            await _context.SaveChangesAsync();


            return RedirectToAction(nameof(Index));
        }
    }
}
```

Explanation:

- private readonly AppDbContext _context;: Declares a private, read-only variable to hold the database context.
- public BooksController(AppDbContext context): The constructor. ASP.NET Core automatically passes the registered AppDbContext here (Dependency Injection).
- _context = context;: Assigns the injected context to the private variable for use inside the controller methods.
- public async Task<IActionResult> Index(): Defines an asynchronous method to handle requests to the main Books page. async and Task prevent the server from freezing while waiting for the database.
- var books = await _context.Books.ToListAsync();: Queries the database for all rows in the Books table, converts them to a C# List, and waits (await) for the operation to finish.
- return View(books);: Sends the list of books to the HTML View to be displayed.
- [HttpGet("New")]: Maps this method to a GET request at the URL /Books/New.
- public IActionResult Create() { return View(); }: Simply returns the empty HTML form for adding a book.

- [HttpPost("New")]: Maps this method to a POST request (form submission) at /Books/New.

- [ValidateAntiForgeryToken]: A security measure that ensures the form submission came from your actual website, preventing Cross-Site Request Forgery attacks.

- public async Task<IActionResult> Create(Book book): Accepts the form data bound to a Book object.

- if (!ModelState.IsValid): Checks if the submitted data breaks any rules defined by Data Annotations in Book.cs.

- return View(book);: If validation fails, it returns the same View along with the user's data so they can fix their errors without losing progress.

- _context.Books.Add(book);: Tells Entity Framework to track this new book object for insertion.

- await _context.SaveChangesAsync();: Actually executes the SQL INSERT command to save the book into the database permanently.

- return RedirectToAction(nameof(Index));: Redirects the user back to the list of books after a successful save.

- public async Task<IActionResult> Edit(int id): Defines the method to load the edit form for a specific book ID.

- var book = await _context.Books.FindAsync(id);: Asks the database to find a specific book by its Primary Key (id).

- if (book == null) return NotFound();: If the database can't find the ID, it returns an HTTP 404 Not Found error.

- public async Task<IActionResult> Edit(int id, Book updatedBook): Accepts the URL ID and the submitted form data.

- if (id != updatedBook.Id) return BadRequest();: A security check ensuring the ID in the URL matches the hidden ID in the submitted form.

- _context.Update(updatedBook);: Tells Entity Framework that this object has been modified and needs to be updated.
- await _context.SaveChangesAsync();: Executes the SQL UPDATE command in the database.

Section 6: Views/Books/Create.cshtml & Edit.cshtml (UI & Validation)

HTML

```
@model MyMvcApp.Models.Book

<h2>Add a Book</h2>

<form asp-action="Create" method="post">

    <div asp-validation-summary="ModelOnly" class="text-danger mb-3"></div>


    <div class="mb-3">

        <label asp-for="Title"></label>

        <input asp-for="Title" class="form-control" />

        <span asp-validation-for="Title" class="text-danger"></span>

    </div>


    <button type="submit" class="btn btn-success">Save</button>

</form>


@section Scripts {

    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}

}
```

Explanation:

- @model MyMvcApp.Models.Book: Tells the view to expect a Book object. This enables strongly-typed Tag Helpers.
- <form asp-action="Create" method="post">: Generates an HTML <form>. The asp-action="Create" tells it to submit data to the Create method in the controller.
- <div asp-validation-summary="ModelOnly"...> : Displays a bulleted list of general, form-level validation errors (if any exist) at the top of the form.
- <label asp-for="Title"></label>: Generates an HTML <label> specifically tied to the Title property of the model.
- <input asp-for="Title" class="form-control" />: Generates a text input. It automatically populates with the model's value (useful when returning the user to the form after an error).
- : This is where the specific error message (e.g., "The Title field is required.") defined in Book.cs will appear if validation fails for this specific field.
- @section Scripts { ... }: Injects JavaScript into the page layout.
- @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}: Loads Microsoft's jQuery validation scripts. This enables **client-side validation**, meaning the browser will catch errors (like a blank title) and show messages instantly *before* the form is even sent to the server.